رِزمة · RIZMA

الدليل التقني الشامل — بنية المشروع ونوعه والمكتبات والقرارات، حتى المرحلة الحالية

Feature-First + Layered | Offline-First · Local-First | Flutter 3.38 · Dart 3.10 | Riverpod · go_router · Drift (SQLite) | 231 اختبارًا آليًا

لمن هذا الملف؟ لأي شخص يريد أن يفهم المشروع كاملًا دون أن يقرأ الكود سطرًا بسطر: عضو جديد في الفريق، مراجع، لجنة، أو أنت بعد أشهر. يشرح **ماذا** بُني، و**كيف** رُتّب، و**لماذا** اختيرت كل تقنية. للتفاصيل الدقيقة لكل سطر كود ارجع إلى `docs/line_by_line/` (30 ملف PDF).

المحتويات

1. ما هو التطبيق ونوعه
2. القرارات التقنية الكبرى ولماذا
3. لماذا Riverpod وليس GetX؟
4. البنية المعمارية (Architecture)
5. خريطة المجلدات والملفات
6. طبقة التطبيق `lib/app`
7. النواة `lib/core`
8. الميزات `lib/features`
9. قاعدة البيانات ونموذج البيانات
10. نظام المحتوى (الكتالوج والدروس)
11. المنطق النقي: المحرّكات والحسابات
12. المكتبات المستعملة: وظيفتها ولماذا
13. الواجهة: RTL والتصميم وإمكانية الوصول
14. الأمان والخصوصية
15. الاختبارات والجودة
16. البناء والتشغيل
17. التوثيق
18. المرحلة الحالية وما بعدها
19. قاموس المصطلحات

1) ما هو التطبيق ونوعه

رزمة تطبيق تعليمي لطلاب الصفوف 1–12 في فلسطين، تجربته الأولى في غزة. يحوّل الرزم التعليمية المدرسية إلى رحلة تعلّم تفاعلية: **رزمة ← شرح ← تدريب ← اختبار ← خطة مراجعة**، ويعمل كاملًا دون إنترنت.

السؤال	الجواب
نوع التطبيق	تطبيق جوال (Cross-Platform) مكتوب مرة واحدة بـ Flutter: أندرويد أولًا، ثم iOS، ويعمل على الويب (Chrome) للتطوير فقط. **متعدد المنصات**
نمط التشغيل	**Offline-First / Local-First:** كل البيانات على الجهاز (قاعدة SQLite)، وكل المحتوى مضمَّن داخل التطبيق. لا خادم حاليًا ولا حساب على الإنترنت.
نمط المستخدمين	**جهاز واحد، عدة طلاب:** هاتف العائلة يحمل ملفًا لكل أخ، مع رمز سري اختياري. لا بريد ولا رقم هاتف.

اللغة والاتجاه	العربية فقط، والاتجاه من اليمين إلى اليسار (RTL)، بخط Cairo.
الحجم الحالي	137 ملف Dart (نحو 20 ألف سطر) في 10 ميزات، و24 ملف اختبار (231 اختبارًا)، و35 شاشة من Figma، و10 دروس محتوى بـ164 سؤالًا (مسودة تنتظر مراجعة معلم).
هوية الحزمة	`com.rizma.rizma` · الاسم الظاهر `Rizma` · الإصدار `1.0.0+1`

2) القرارات التقنية الكبرى ولماذا

القرار	ماذا اخترنا	لماذا
إطار العمل	**Flutter** (لغة Dart)	كود واحد لأندرويد وiOS والويب؛ أداء قريب من الأصلي لأنه يرسم الواجهة بنفسه؛ دعم ممتاز للعربية وRTL؛ يسهّل مطابقة تصميم Figma بالبكسل؛ مناسب لفريق صغير. البدائل: Kotlin وSwift (مشروعان منفصلان) أو React Native (اعتماد أكبر على جسور أصلية).
نمط التشغيل	**Offline-First** بقاعدة محلية ومحتوى مضمَّن	الإنترنت في غزة منقطع أو ضعيف؛ يجب ألا يضيع تقدّم الطالب أبدًا. كل عملية تُكتب محليًا أولًا، والمزامنة (لاحقًا) طبقة فوقها لا شرط لعمل التطبيق.
إدارة الحالة	**Riverpod 2.6** (بلا توليد كود)	مزوّدات آمنة الأنواع، تُستبدل بسهولة في الاختبارات، تدعم البثّ من قاعدة البيانات، ولا تعتمد على كائنات عامة مخفية. انظر القسم 3.
التنقّل	**go_router**	من فريق Flutter الرسمي؛ مسارات نصية (روابط مباشرة)، وإعادة توجيه مركزية (مثلًا: لا ملف نشط ← شاشة الإقلاع)، وحركات انتقال مخصّصة.
التخزين	**Drift** فوق **SQLite**	قاعدة علائقية حقيقية بمفاتيح وقيود وحذف متسلسل ومعاملات؛ استعلامات آمنة الأنواع تُولَّد وقت البناء؛ بثّ تغييرات (Streams) يحدّث الواجهة تلقائيًا؛ ترحيل منظّم بين الإصدارات. البدائل: sqflite (SQL نصّي بلا أمان أنواع)، Hive/Isar (مفتاح-قيمة/وثائق بلا علاقات ولا معاملات بنفس القوة).
المحتوى	ملفات **JSON** داخل التطبيق	يعمل دون إنترنت، ويكتبه المؤلف بلا أدوات، وإضافة درس = إضافة ملف (لا برمجة). مولّد يفحص كل سؤال قبل الكتابة.
البنية	**Feature-First** مع **طبقات**	كل ميزة في مجلدها (سهلة الإيجاد والحذف)، وداخلها فصل بين الواجهة والحالة والمستودع والقاعدة. انظر القسم 4.
«يُحسب ولا يُخزَّن»	النجوم والسلسلة والنسب والشارات والرصيد مشتقة	رقم مخزَّن قد يتناقض مع المحاولات؛ الاشتقاق يضمن أن كل الأرقام صحيحة دائمًا ولا تحتاج ترحيلًا عند تغيير القواعد.

النطق	محرّك الجهاز (**flutter_tts**)	لا ملفات صوت تكبّر التطبيق، ويعمل دون إنترنت غالبًا؛ وتسجيلات بشرية ممكنة لاحقًا.
الرسوم والأيقونات	SVG تُلوَّن وقت العرض + رسوم بالكود	حجم صغير، حادّة بأي دقة، ولون واحد لكل ملف يُعاد استعماله بألوان التصميم.

3) لماذا Riverpod وليس GetX؟

(جواب سؤالك المكتوب في `lib/main.dart`)

GetX مكتبة شاملة شائعة: حالة وتنقّل وحقن تبعيات وأدوات في حزمة واحدة، وكتابتها مختصرة. لكنها لا تناسب هذا المشروع لهذه الأسباب:

المعيار	GetX	Riverpod (ما اخترناه)
الاختبارات	الحالة في كائنات عامة (`Get.put` / `Get.find`) تعيش خارج شجرة الواجهة؛ عزلها بين الاختبارات واستبدالها يدوي وعرضة للتسرّب.	كل اختبار ينشئ `ProviderScope` جديدًا ويستبدل ما يريد بسطر `overrideWithValue` . نستبدل بهذا: القاعدة (بقاعدة في الذاكرة)، المحتوى، النطق (`FakeSpeech`)، العشوائية (بذرة ثابتة)، والساعة. 231 اختبارًا قائمة على هذا.
أمان الأنواع	`Get.find<T>()` يفشل وقت التشغيل إن نُسي التسجيل.	المزوّد متغيّر عادي؛ المترجم يكشف أي استعمال خاطئ قبل التشغيل.
البيانات المتدفقة	يحتاج ربطًا يدويًا للـ Streams.	`StreamProvider` يربط بث Drift بالواجهة مباشرة: أي كتابة في القاعدة تحدّث الشاشات تلقائيًا.
دورة الحياة	يعتمد على قواعد GetX الخاصة لحذف الكائنات.	`autoDispose` يحرّر المزوّد حين لا تستعمله أي شاشة، و `family` يخزّن نسخة لكل معرّف (لكل درس، لكل طالب).
التنقّل	نظام تنقّل خاص بـ GetX.	نستعمل go_router الرسمي (روابط مباشرة، إعادة توجيه مركزية) وهو مستقل عن إدارة الحالة.
الصيانة والمجتمع	مكتبة تجمع مسؤوليات كثيرة، ويحذّر منها كثير من فرق Flutter للمشاريع الطويلة لأنها تخفي التبعيات.	من مؤلف `provider` (الذي أوصى به فريق Flutter)، وثائق قوية ومجتمع كبير.

الخلاصة: نريد مشروعًا يكبر إلى 12 صفًا ويُختبر جيدًا ويفهمه أعضاء جدد. Riverpod يجعل التبعيات **صريحة وقابلة للاستبدال**، وهذا ما مكّننا من اختبار كل شيء. GetX أسرع للبداية لكنه يصعّب ذلك.

لماذا Riverpod 2.6 لا 3، وبلا توليد كود؟ الإصدار 2.6 مستقر ومتوافق مع قيود حزم المشروع الحالية (خصوصًا تعارض نسخة `meta` مع حزم التوليد)، وتجنّب توليد الكود للمزوّدات يبقي البناء أبسط. الترقية ممكنة لاحقًا دون تغيير البنية.

4) البنية المعمارية (Architecture)

نستعمل **Feature-First** (التقسيم بحسب الميزة) مع **Layered Architecture** (طبقات) داخل كل ميزة. والقاعدة الذهبية: **التبعية تتجه للأسفل فقط** — الواجهة تعرف الحالة، والحالة تعرف المستودع، والمستودع يعرف القاعدة؛ ولا يعرف أي مستوى أسفل شيئًا عن الواجهة.

الواجهة (Presentation) — Screens & Widgets — تعرض الحالة وتستقبل اللمس، بلا منطق عمل

▼ تقرأ عبر `ref.watch` وتطلب عبر `ref.read`

الحالة (State — Riverpod Providers) — تجمع المصادر وتشتق ما تعرضه الشاشة (HomeState, ProgressState...)

▼

المنطق النقي (Domain Logic) — RoundEngine, ReviewPlanner, ExamPrep, PinHasher... — دوال بلا واجهة ولا قاعدة

▼

المستودعات (Repositories) — ProfileRepository, LearningRepository, WardrobeRepository, ContentRepository — عمليات العمل والمعاملات

▼

البيانات (Data — DAOs + Drift + SQLite / JSON Assets) — جداول، استعلامات، بث تغييرات، وملفات المحتوى

لماذا هذه البنية؟

- **قابلية الاختبار:** المنطق النقي يُختبر بلا Flutter إطلاقًا (محرّك الأسئلة 21 اختبارًا يعمل في أقل من ثانية).
- **قابلية الاستبدال:** لو أضفنا خادمًا لاحقًا يتغيّر المستودع فقط؛ الشاشات لا تعلم.
- **سهولة الفهم:** لتعرف «أين تُحسب النجوم؟» تذهب لملف واحد معروف.
- **Feature-First:** كل ميزة (التقدّم، الخزانة...) في مجلد واحد فيه شاشاتها ومزوّداتها ومنطقها؛ إضافة ميزة لا تبعثر الملفات.

مثال حيّ: ماذا يحدث حين ينهي الطالب جولة تدريب؟

RoundScreen ← **RoundEngine.advance** ← **LearningRepository.recordRound** ← **معاملة Drift** ← **SQLite**

1. الشاشة تحتفظ بـ `RoundState` وتستدعي `RoundEngine.advance` (دالة نقية) فتحصل على النتائج.
2. تستدعي `LearningRepository.recordRound`، فيفتح **معاملة واحدة**: يحسب النجوم الجديدة (دون تكرار ما نيل سابقًا)، يحدّث إتقان كل مفهوم بمتوسط متحرّك، يُدخل المحاولة بتفاصيل كل سؤال (JSON)، وينجز جزء الدرس.
3. Drift يبثّ التغيير، فتتحدّث **تلقائيًا** الرئيسية و«تقدّمي» ورصيد الخزانة وشارة النجوم — دون أي استدعاء يدوي.
4. الشاشة تنتقل إلى «نهاية الجولة» بمعرّف المحاولة، وتقرأ تفاصيلها من القاعدة (فلا تضيع لو أُغلق التطبيق).

5) خريطة المجلدات والملفات

```
Rizma/
├ lib/
│  ├ main.dart              ← نقطة البداية: ProviderScope + RizmaApp
│  ├ app/                   ← التطبيق: MaterialApp، التبويبات، الانتقالات، المسارات
│  ├ core/                  ← المشترك بين الميزات
│  │  ├ db/                 ← Drift: الجداول، DAOs، المزوّد، الترحيل
│  │  ├ theme/              ← الألوان، الخطوط، ألوان الشخصية والمواد
│  │  ├ strings/            ← كل النصوص العربية الظاهرة
│  │  ├ utils/              ← رمز سري، رمز طالب، رمز صف، سلسلة أيام، اهتزاز
│  │  ├ widgets/            ← مكوّنات واجهة مشتركة (أزرار، بطاقات، كسور، رسوم…)
│  │  └ constants/          ← مسارات الأصول
│  └ features/              ← الميزات (كل ميزة: شاشات + مزوّدات + منطق + مستودع)
│     ├ launch/  onboarding/  profiles/  home/  account/
│     ├ catalog/  content/  learning/  progress/  wardrobe/  kids/
├ assets/
│  ├ content/               ← catalog_grade5.json + lessons/*.json
│  ├ icons/ui/              ← 60 أيقونة SVG من Figma
│  ├ images/brand/          ← الشخصية والشعار
│  └ fonts/                 ← Cairo (4 أوزان)
├ test/                    ← 24 ملف اختبار + helpers/
├ tools/content/           ← مولّد المحتوى وفاحصه (Node.js)
├ design_spec/figma/       ← مواصفات Figma المصدّرة ولقطات الشاشات
├ branding/                ← مصدر الأيقونة (SVG)
├ android/ ios/ web/       ← ملفات المنصات (الأيقونة، شاشة البداية، الاسم)
└ docs/                    ← الشرح (md + PDF) والشرح سطرًا بسطر
```

6) طبقة التطبيق `lib/app`

الملف	دوره
`rizma_app.dart`	`MaterialApp.router` : اللغة العربية (`Locale('ar')`) ومترجمات Flutter الرسمية للعربية، الثيم، تمرير مرن بلا توهّج. وفي `builder` : يحصر العرض في 480 بكسل على الشاشات العريضة، ويضرب حجم الخط الذي اختاره الطالب في مقياس النظام (فلا نلغي إعداد إمكانية الوصول).
`router.dart`	تعريف كل المسارات بـ go_router، وإعادة توجيه مركزية: أي شاشة تحتاج ملفًا نشطًا وتُفتح بلا ملف ← شاشة الإقلاع؛ جزء درس مجهول ← صفحة الدرس.
`routes.dart`	ثوابت المسارات ودوال بنائها (`AppRoutes.lesson(id)` …)، منفصلة كي لا تعتمد الشاشات على الراوتر.
`page_transitions.dart`	حركتا انتقال: «تبويب» (تلاشٍ سريع 180ms) و«دفع» (تلاشٍ وانزلاق من جهة القراءة 260ms).
`tab_navigation.dart`	ما يحدث عند ضغط تبويب: الرئيسية · المواد · تقدّمي · حسابي.

منطق الإقلاع (`launch_plan.dart`)

دالة نقية مختبَرة: لا ملفات ← الترحيب؛ ملف واحد بلا رمز سري ← الرئيسية مباشرة؛ غير ذلك ← «من يتعلّم الآن؟».

7) النواة `lib/core`

المجلد	المحتوى ولماذا
`db/`	`AppDatabase` (8 جداول، 4 DAOs، الإصدار 2 مع ترحيل)، `appDatabaseProvider` و `clockProvider` (الساعة قابلة للاستبدال في الاختبار). تفعيل `PRAGMA foreign_keys` ليعمل الحذف المتسلسل.
`theme/`	`AppColors` و `AppTextStyles` بالقيم الحرفية من Figma، و `ProfilePalette` (5 ألوان للشخصية)، و `SubjectTone` (6 ألوان للمواد).
`strings/`	`ArStrings` (عام) و `ArLearn` (التعلّم) و `ArProgress` (التقدّم والتنزيلات والخزانة) و `ArKids` (الصغار والمراهقون). **قاعدة:** لا نص ظاهر خارجها، والجمع والمثنى العربيان بدوال.
`utils/`	`PinHasher` (تجزئة الرمز السري وقفل المحاولات)، `ClassCode` (`RZ-XXXX`)، `StudentCode` و `currentStreak` (الأيام المتتالية)، `Haptics` (اهتزاز خفيف).
`widgets/`	19 مكوّنًا مشتركًا: `RizmaButton` (6 أنواع)، `MathText`، `BottomNav`، `ScreenTopBar` (كسور وخانات داخل النص)، `Illustration`، `FractionBar`، `Mascot` (فواكه بالكود)، `AppIcon` و `MascotWithItems` (الشخصية بإكسسواراتها)، `ProgressBar` و `PressableScale`، `PinPad`، `ProgressRing` و `FadeSlideIn` (السلاسة)...

8) الميزات `lib/features`

الميزة	الشاشات	المسؤولية
`launch`	الإقلاع	يقرر الوجهة عند فتح التطبيق.
`onboarding`	01 الترحيب، 02 الصف، 26 إعداد الملف	إنشاء ملف الطالب على خطوتين (مسودة في الذاكرة حتى الإنهاء).
`profiles`	25 من يتعلّم الآن، 27 الرمز السري	`ProfileRepository` : إنشاء/حذف/تفعيل، الرمز السري و«نسيت الرمز؟» برمز الطالب، الملف النشط.
`home`	03/28 الرئيسية، 20 (1–4)، 22 (9–12)	`HomeState` و `ExamPrep` مشتقّان؛ تختار الرئيسية بحسب الصف.
`account`	19 حسابي، 29 رمز الطالب، 18 التنزيلات	التفضيلات لكل طالب (حجم الخط، الأصوات...) في جدول الإعدادات بمفاتيح `pref.<id>.*` .
`catalog`	—	قراءة كتالوج الصف (المواد والوحدات والدروس والمفاهيم).
`content`	—	نماذج الأسئلة ومحتوى الدرس، قراءة ملفات الدروس، خدمة النطق.
`learning`	04–16، 23، 30–35	التصفّح، أجزاء الدرس، محرّك الأسئلة، شاشات الجولة، النتيجة، خطة المراجعة، تسجيل التقدّم.
`progress`	17 تقدّمي	`ProgressState` مشتق بالكامل.
`wardrobe`	24 الخزانة	العناصر، الشراء بمعاملة، اللبس، تغيير اللون.
`kids`	21 لعبة العدّ	توليد أسئلة العدّ ولعبها بمحرّك الجولة.

9) قاعدة البيانات ونموذج البيانات

الجدول	المفتاح	ماذا يحفظ
`profiles`	`id` (UUID)	الاسم (1–40)، الصف (1–12 بقيد)، لون الشخصية، تجزئة الرمز السري وملحه (أو null)، رمز الطالب (فريد)، رمز الصف، تواريخ الإنشاء وآخر نشاط.
`lesson_progress`	(طالب، درس)	حالة الدرس (لم يبدأ/قيد التعلّم/مكتمل) ونسبته وآخر تحديث.
`lesson_part_progress`	(طالب، درس، جزء)	الأجزاء المنجزة؛ **وجود الصف يعني «أُنجز»**.
`attempts`	`id` تلقائي	كل جولة تدريب أو اختبار: النشاط، النتيجة، الكل، النجوم المكتسبة، التلميحات، وتفاصيل كل سؤال (JSON).
`concept_mastery`	(طالب، مفهوم)	إتقان المفهوم 0..1 (متوسط متحرّك).

`owned_items`	(طالب، عنصر)	مشتريات الخزانة وتكلفتها وقت الشراء.
`sync_queue`	`id` تلقائي	ما ينتظر الرفع للخادم (يمتلئ الآن ولا يُرسل).
`app_settings`	`key`	مفتاح/قيمة: الملف النشط، وتفضيلات كل طالب (`pref.<id>.fontSize`، المرتدى، الدروس المحفوظة...).

- **العزل بين الإخوة:** كل جدول نشاط مربوط بالملف بمفتاح أجنبي، و**حذف الملف يحذف كل ما يخصّه** (حذف متسلسل) في معاملة.
- **الترحيل:** الإصدار 1 ← 2 أضاف أجزاء الدرس والخزانة وعمودَي التلميحات والتفاصيل دون فقد بيانات.
- **المشتق (لا يُخزَّن):** مجموع النجوم، الأيام المتتالية، نسب الوحدات والمواد، الشارات، رصيد الخزانة، الجاهزية للامتحان.
- **DAOs:** `ProfilesDao` و`ActivityDao` و`SettingsDao` و`WardrobeDao` — كل استعلام مفلتر بمعرّف الطالب.

10) نظام المحتوى (الكتالوج والدروس)

- **الكتالوج** 6 : `assets/content/catalog_grade5.json` مواد · 19 وحدة · 55 درسًا · 86 مفهومًا.
- **ملف الدرس** `pack` : `assets/content/lessons/<id>.json` (صفحات) · `explanation` (خطوات) · `words` · `video` · `summary` · `quiz` · `practice` ·. الجزء الغائب = غير متوفر (لا نختلقه).
- **ستة أنواع أسئلة** في صنف مغلق (`sealed`): اختيار، صح/خطأ، سحب وإفلات، توصيل، ترتيب، استمع واختر. كل نوع يصحّح نفسه بأرقام العناصر **الأصلية**، فالخلط وقت العرض لا يكسر التصحيح.
- **صيغة الرياضيات:** `{3/4}` كسر، `{?}` خانة، `{?/8}` كسر بخانة — يرسمها `MathText` بودجتات داخل السطر.
- **المولّد** (Node.js) `tools/content/` : يكتب الدروس بدوال مختصرة ويفحص كل سؤال (المفهوم موجود، الجواب بين الخيارات، القالب فيه خانة...) ويفشل بخطأ واضح بدل كتابة ملف معطوب.

> **تنبيه:** المحتوى الحالي (10 دروس، 164 سؤالًا) **مسودة** ولا يُنشر قبل اعتماد معلم، كما ينص المقترح.

11) المنطق النقي: المحرّكات والحسابات

الوحدة	القواعد
`RoundEngine`	**تدريب:** 8 أسئلة مخلوطة؛ الخطأ يُظهر تلميحًا ويسمح بالمحاولة؛ بعد نفاد التلميحات يُكشف الحل خطوة بخطوة ويُحتسب خطأ. **اختبار:** حتى 10 بترتيب المعلم، محاولة واحدة بلا ردّ. **القيم:** صح أول مرة = 3 نجوم وإتقان 1؛ بعد تلميح = 2 و0.5؛ خطأ = 0. «كل ما سبق» لا تُخلط، والترتيب والتوصيل لا يبدآن صحيحين.
الإتقان	أول قياس يؤخذ كما هو، ثم `القديم × 0.4 + الجديد × 0.6` .
تقييم النتيجة	3 نجوم بحسب نسبة الإتقان: 30% · 60% · 90%. الاختبار يُنجز الجزء عند 60% فأكثر.
منع تكرار النجوم	نجوم الجولة − ما نِيل سابقًا من النشاط نفسه (يُكافأ التحسّن فقط). لعبة العدّ بسقف إجمالي 60.
`ReviewPlanner`	أضعف مفهوم دون 80% (غير المقيس = 0) ← شرح ← فيديو ← حتى 5 تمارين مخصّصة ← إعادة الاختبار.
`ProgressState`	التقدّم العام من الدروس ذات المحتوى فقط؛ نشاط الأسبوع (يبدأ بالسبت)؛ 5 شارات مشتقة.
`ExamPrep`	الجاهزية = متوسط آخر 3 اختبارات؛ أضعف 3 مفاهيم؛ خطة أسبوع قصيرة.
`WardrobeRepository`	الرصيد = النجوم − المصروف؛ الشراء معاملة بفحوص (مملوك؟ مقفل؟ يكفي؟).

12) المكتبات المستعملة: وظيفتها ولماذا

من `pubspec.yaml` . لكل مكتبة: وظيفتها، أين تُستعمل، ولماذا هي بالذات.

Flutter SDK + Dart

`sdk: ^3.10.1`

الوظيفة: إطار الواجهة واللغة. يرسم كل بكسل بنفسه (محرّك Impeller/Skia) فتبدو الواجهة متطابقة على كل الأجهزة.

لماذا: كود واحد لكل المنصات، RTL ممتاز، أداء عالٍ على الأجهزة الضعيفة، وإعادة تحميل فورية أثناء التطوير.

flutter_localizations

`sdk: flutter`

الوظيفة: ترجمات Flutter الرسمية لعناصر النظام (الحوارات، أزرار القوائم، التواريخ) واتجاه RTL.

أين: `rizma_app.dart` (المترجمات الثلاث: Material وWidgets وCupertino).

لماذا: بدونها تظهر نصوص النظام بالإنجليزية ولا ينعكس الاتجاه في بعض العناصر.

flutter_riverpod

`^2.6.1`

الوظيفة: إدارة الحالة وحقن التبعيات: `autoDispose` ، `family` ، `StreamProvider` ، `FutureProvider` ، `Provider` .

أين: في كل ميزة (`*_providers.dart`) وفي `ProviderScope` (`main.dart`).

لماذا: انظر القسم 3 — قابلية الاستبدال في الاختبارات، أمان الأنواع، ربط بث القاعدة بالواجهة.

go_router

`^17.5.0`

الوظيفة: التنقّل بمسارات نصية، إعادة توجيه، معاملات المسار والاستعلام، وصفحات بحركات مخصّصة.

أين: `lib/app/router.dart` و `routes.dart` ؛ والشاشات تستعمل `context.push/go/pop` .

لماذا: الحل الرسمي من فريق Flutter؛ `redirect` مركزي يحرس الشاشات التي تحتاج ملفًا نشطًا؛ روابط مثل `/learn/round/math.u3.l3?concept=…` تفتح الحالة مباشرة (وتسهّل الاختبار).

drift + drift_flutter

`2.31.x + 0.2.x`

الوظيفة: قاعدة SQLite بواجهة Dart آمنة الأنواع: جداول وقيود ومفاتيح أجنبية ومعاملات وبث تغييرات وترحيل. `drift_flutter` يجهّز ملف القاعدة على الجهاز (ويدعم الويب بـ WASM).

أين: `lib/core/db/` .

لماذا: بياناتنا علائقية (طالب ← تقدّم ← محاولات ← إتقان)، ونحتاج معاملات (الجولة والشراء) وحذفًا متسلسلًا. والبثّ يحدّث الشاشات تلقائيًا. الإصدار مثبّت ضمن نطاق ضيّق لتوافقه مع حزم التوليد.

flutter_svg

`^2.3.0`

الوظيفة: عرض ملفات SVG، مع استبدال الألوان وقت العرض.

أين: `AppIcon` (60 أيقونة من Figma تُلوَّن بـ `ColorFilter`) و `Mascot` (الشخصية بلون الطالب عبر `ColorMapper`).

لماذا: حادّة بأي حجم وصغيرة الحجم؛ ملف واحد يخدم كل الألوان الخمسة للشخصية.

flutter_tts

`^4.2.5`

الوظيفة: تحويل النص إلى كلام بمحرّك الجهاز (سرعة عادية وبطيئة).

أين: `SpeechService` (خلف واجهة مجرّدة): نطق كلمات الإنجليزية، «استمع واختر»، قراءة الشرح والتحية بالعربية، سؤال لعبة العدّ.

لماذا: بلا ملفات صوت تكبّر التطبيق، ويعمل دون إنترنت عادةً. الفشل (لا صوت عربي على الجهاز) يعيد `false` فتظهر رسالة بدل انهيار.

crypto

`^3.0.7`

الوظيفة: خوارزميات التجزئة (SHA-256).

أين: `PinHasher` : تجزئة الرمز السري مع ملح عشوائي 16 بايت ومقارنة بزمن ثابت.

لماذا: لا يُحفظ الرمز السري نصًّا أبدًا. (انظر حدود هذه الحماية في القسم 14.)

uuid

`^4.6.0`

الوظيفة: توليد معرّفات فريدة عالميًا.

أين: `ProfileRepository` : معرّف كل ملف طالب.

لماذا: عند المزامنة لاحقًا لن يتعارض معرّف ملف من جهاز مع آخر (بعكس الأرقام المتسلسلة).

share_plus

`^13.3.1`

الوظيفة: فتح نافذة المشاركة في النظام.

أين: شاشة «رمز الطالب»: مشاركة الرمز ليُحفظ (عبر واتساب أو ملاحظة).

لماذا: واجهة موحّدة لكل المنصات، ومزوّدة كي تُستبدل في الاختبارات.

path_provider · cupertino_icons

`^2.1.6 · ^1.0.8`

ملاحظة صادقة كلاهما مُدرج في `pubspec.yaml` لكنه **غير مستعمل مباشرة** في كودنا: `cupertino_icons` من قالب Flutter الافتراضي، و `path_provider` يستعمله `drift_flutter` داخليًا لتحديد مكان ملف القاعدة. يمكن حذفهما من القائمة المباشرة لاحقًا دون أثر.

أدوات التطوير (dev_dependencies)

الأداة	وظيفتها ولماذا

`drift_dev` + `build_runner`	يولّدان كود قاعدة البيانات (ملفات `*.g.dart`) من تعريف الجداول: `dart run build_runner build --delete-conflicting-outputs`. التوليد يمنحنا أمان الأنواع دون كتابة SQL يدويًا.
`flutter_lints`	قواعد التحليل الثابت الرسمية؛ `flutter analyze` يجب أن يبقى بلا ملاحظات.
`flutter_test`	اختبارات الوحدة والواجهة.

ما لم نضفه عمدًا

- **GetX / Bloc:** Riverpod يغطي الحاجة (القسم 3).
- **Freezed / json_serializable:** النماذج قليلة وبسيطة وغير قابلة للتغيير يدويًا؛ تجنّبنا خطوة توليد إضافية.
- **مشغّل فيديو / Firebase / HTTP:** لا فيديو مضمّن ولا خادم بعد؛ تُضاف مع مراحلها.
- **مكتبة رسوم بيانية:** الحلقة والأعمدة بسيطة فرسمناها بأنفسنا (أخف وأدق مطابقة للتصميم).

13) الواجهة: RTL والتصميم وإمكانية الوصول

- **RTL:** أول عنصر في الصف يظهر على اليمين؛ نستعمل `EdgeInsetsDirectional` و `AlignmentDirectional` دائمًا. الرياضيات والإنجليزية تُرسم LTR داخل السياق العربي.
- **مطابقة Figma:** القيم (ألوان، أحجام، زوايا، مسافات) منقولة حرفيًا من مواصفات مصدّرة بإضافة Figma كتبناها (`tools/figma_export_plugin`)، و35 شاشة نُفّذت.
- **Cairo:** أربعة أوزان مضمّنة (400–800) فلا تعتمد على خطوط الجهاز.
- **إمكانية الوصول:** أوصاف `Semantics` لكل زر أيقوني، أهداف لمس ≥ 44 بكسل، احترام تكبير الخط من النظام × اختيار الطالب، بدائل للسحب (النقر في «اسحب وأفلت»).
- **السلاسة:** انتقالات مخصّصة، ظهور متدرّج للقوائم، تصغير خفيف عند الضغط، واهتزاز لطيف.
- **الشاشات العريضة:** العرض محصور بـ 480 بكسل مثل هاتف.

14) الأمان والخصوصية

الجانب	كيف تعاملنا معه
بيانات الطفل	لا بريد ولا رقم هاتف ولا حساب على الإنترنت؛ الاسم والصف فقط على الجهاز.
الرمز السري	4 أرقام، يُحفظ تجزئة SHA-256 بملح عشوائي لكل ملف، مقارنة بزمن ثابت، وقفل 30 ثانية بعد 5 محاولات. **حدّه:** قفل خفيف بين الإخوة لا حماية قوية (10,000 احتمال فقط) — موثّق في الكود.
رمز الطالب	`RZ-XXXX` بأحرف بلا التباس (لا O/1/I/L/0)، لاستعادة الرمز السري ونقل التقدّم لاحقًا، ولا يكشف هوية.
سلامة البيانات	المعاملات تمنع الكتابة الجزئية؛ الحذف المتسلسل لا يترك بقايا؛ الجولة المتروكة لا تُحفظ.
الشبكة	لا اتصال شبكي في التطبيق حاليًا.

15) الاختبارات والجودة

- **231 اختبارًا** في 24 ملفًا: وحدة (المحرّكات والحسابات والمستودعات على قاعدة في الذاكرة)، وواجهة (شاشات ورحلات كاملة)، وتكامل (إنشاء ملف ← الرئيسية...).
- **أداة `scenario` :** تشغّل التطبيق كاملًا على قاعدة في الذاكرة مع محتوى ونطق مزيّف وساعة وعشوائية مضبوطة، ثم تنظّف.
- **فحص الضغط للتخطيط:** كل سؤال مكتوب في المحتوى وكل شاشة تُعرض على 320×568 بخط 1.0× و1.5× دون أي فيض — كشف مشاكل حقيقية وأُصلحت.
- `flutter analyze` بلا ملاحظات، و `dart format` على كل الملفات.
- **فخاخ موثّقة** في `docs/explanations/12_testing.md` (Drift داخل الزمن الوهمي، السحب في الاختبار...).

16) البناء والتشغيل

```
flutter pub get                                          # تنزيل المكتبات
dart run build_runner build --delete-conflicting-outputs   # بعد أي تعديل على الجداول
flutter run                                              # تشغيل على جهاز/محاكي
flutter test --concurrency=2                             # الاختبارات (تزامن 2 لتوفير الذاكرة)
flutter analyze                                          # التحليل الثابت
flutter build apk --release --target-platform android-arm64   # APK للجوال
node tools/content/build_catalog.js && node tools/content/build_lessons.js  # توليد المحتوى
```

على جهاز ذاكرته محدودة نبني الـ APK بإعدادات Gradle خفيفة (ذاكرة 1GB، عامل واحد، بلا daemon) ولمعمارية arm64 فقط (تغطي كل الهواتف الحديثة) لتصغير الحجم (~23MB).

17) التوثيق

المكان	المحتوى
`docs/explanations/` + `pdf/`	21 ملف شرح بحسب الموضوع (البنية، القاعدة، كل شاشة، المحرّك، الاختبارات...).
`docs/line_by_line/`	30 ملف PDF: كل مقطع كود + «ماذا يفعل؟» + «لماذا بهذه الطريقة؟»، تغطية كاملة لكل سطر في `lib/` .
`docs/RIZMA_project_overview.pdf`	ملخّص الحالة ومطابقة المقترح.
هذا الملف	الدليل التقني الشامل.

18) المرحلة الحالية وما بعدها

وصلنا: نهاية مرحلة التطوير تقريبًا — كل الشاشات الـ35 تعمل ببيانات حقيقية ومحفوظة محليًا، والتطبيق جاهز للتجربة الداخلية على الجوال.

المتبقّي	كيف سيندمج في البنية
مراجعة المحتوى وتوسيعه (~250 سؤالًا)	ملفات JSON فقط، عبر المولّد والفاحص.
المزامنة مع خادم	عامل يقرأ `sync_queue` ويرسل عند الاتصال؛ طبقة جديدة تحت المستودعات، والشاشات لا تتغيّر.
تنزيل انتقائي للدروس والفيديو	`ContentRepository` يقرأ من مجلد على الجهاز إضافةً إلى الأصول المضمّنة.
كتالوجات الصفوف الأخرى	ملف كتالوج لكل صف يُختار بحسب صف الملف.
الوضع الليلي	ثيم ثانٍ من نفس رموز الألوان.

التجربة الميدانية	المحاولات والإتقان لكل مفهوم محفوظة أصلًا ويمكن تصديرها للقياس.

19) قاموس المصطلحات

المصطلح	المعنى في هذا المشروع
Offline-First	يعمل التطبيق كاملًا دون إنترنت؛ الشبكة (لاحقًا) إضافة لا شرط.
Feature-First	ترتيب الملفات بحسب الميزة لا بحسب النوع.
Provider (مزوّد)	متغيّر Riverpod يقدّم قيمة (قاعدة، حالة، بيانات) لأي شاشة تطلبها، ويمكن استبداله في الاختبار.
Repository (مستودع)	صنف ينفّذ عمليات العمل (إنشاء ملف، تسجيل جولة، شراء) فوق القاعدة.
DAO	صنف استعلامات Drift لمجموعة جداول.
Migration (ترحيل)	ترقية بنية القاعدة من إصدار لآخر دون فقد البيانات.
Transaction (معاملة)	مجموعة كتابات تنجح كلها أو تُلغى كلها.
Stream (بثّ)	قيمة تتجدّد تلقائيًا كلما تغيّرت القاعدة.
Pure function (دالة نقية)	دالة نتيجتها من مدخلاتها فقط، بلا قاعدة ولا واجهة — أسهل ما يُختبر.
RTL	الاتجاه من اليمين إلى اليسار.
Mastery (إتقان)	رقم 0..1 لكل مفهوم يتحدّث مع كل سؤال.
Salt (ملح)	قيمة عشوائية تُخلط مع الرمز السري قبل تجزئته.
Code generation	توليد ملفات Dart آليًا (`*.g.dart`) من تعريفات مختصرة.

رِزمة — لأن التعلّم لا يجب أن ينتظر الإنترنت.

فريق آفاق جديدة · أنس شكشك · صهيب المجايدة · ريم أبو مطلق · محمد أبو صيام